

SLO Engineering, AIOps & Capstone

Design reliability as a discipline — SLIs, SLOs and error budgets — bring AIOps into the loop with anomaly detection and Watchdog, and prove mastery through a full-stack capstone, with diagrams, hands-on labs, solutions, and review questions.

SRE Principles & Error Budgets

SLI · SLO · SLA

SLO Design Patterns

Burn-Rate Monitoring

Static → Dynamic → Anomaly → Forecast

Noise Reduction & RCA

Watchdog & AIOps

Full-Stack Capstone

Prepared for: Vivek Arora

Format: Certification preparation • Full guide • Day 04 (final)

Modules: 10–12 • Labs: 6 • Review questions: 24

Version 1.0 — August 2026

Table of Contents

Day 04 makes reliability a discipline and brings AI into the loop, then ties the whole four-day course together in a capstone. It assumes everything from Days 01–03 (traces, KPIs, percentiles, RUM, Synthetics).

Recap & how Day 04 fits	3
Module 10 — SLO Engineering & Reliability	4
10.1 SRE foundations & the reliability framework	4
10.2 SLI, SLO, SLA	4
10.3 SLO design patterns	6
10.4 Error budgets & burn-rate monitoring	6
10.5 Hands-on labs, use cases & review questions	7
Module 11 — Intelligent Alerting & AIOps	10
11.1 Alert engineering: static to predictive	10
11.2 Incident analytics: noise, correlation, RCA	11
11.3 AIOps & Watchdog	12
11.4 Hands-on lab, use cases & review questions	13
Module 12 — Full-Stack Capstone	15
12.1 Bringing it all together	15
12.2 Capstone scenario & guided solution	16
12.3 Executive reporting	17
Appendix A — Answer keys	18
Appendix B — SRE & alerting cheat sheet	20
Appendix C — Course wrap-up & sources	22

Recap & how Day 04 fits

The final bridge. Day 04 turns measurement into *management*.

Across Days 01–03 you built the full observability stack: distributed tracing and the agent pipeline (Day 01), trace analytics and the four KPI families (Day 02), and business observability plus digital-experience monitoring with RUM and Synthetics (Day 03). You can now see almost everything.

Day 04 answers "so how do we run this reliably, at scale, without drowning in alerts?" **Module 10** makes reliability a discipline: define what "reliable enough" means with SLIs, SLOs, and error budgets, and monitor budget burn. **Module 11** tackles alert quality — moving from noisy static thresholds to anomaly detection and forecasting, and using AIOps/Watchdog to cut noise, correlate incidents, and surface root causes. **Module 12** is the capstone: a single end-to-end scenario that exercises every concept from the whole course, plus how to report it all upward to executives.

The through-line

Day 04's theme is *decision*: observability exists to drive action. An SLO decides whether you ship or freeze; a burn-rate alert decides whether you wake someone; Watchdog decides where to look first; an executive report decides where to invest. Reliability engineering is observability turned into governance.

Scope note & live defaults

Your Day 04 outline lists two modules (10, 11) but frames the day as "3 modules" ending in a "full-stack capstone." Module 12 provides that capstone and is included here. SLO burn-rate thresholds and Watchdog capabilities evolve; confirm against the docs in Appendix C and capture live screenshots where placeholders appear.

MODULE 10

SLO Engineering & Reliability

Turn "is it reliable?" into a measurable, fundable engineering discipline: SLIs, SLOs, SLAs, error budgets, and burn-rate monitoring.

10.1 SRE foundations & the reliability framework

Site Reliability Engineering (SRE) is the practice of applying engineering rigour to operations, treating reliability as a feature with a measurable target rather than an aspiration. Its central insight: *100% reliability is the wrong target* — it is impossibly expensive and users can't tell the difference above a certain point. Instead, you pick a deliberate target and manage to it.

Definition — Core SRE principles

Key tenets: **reliability is the most important feature** (an unreliable product's other features don't matter); **set a target below 100%** and use the gap as budget; **error budgets align incentives** (dev wants to ship, ops wants stability — the budget is the shared currency); **toil is the enemy** (automate repetitive operational work); and **blameless postmortems** (learn from incidents without punishing people).

The **reliability framework** connects three layers: you *measure* reliability with an SLI, *target* it with an SLO, and (optionally) *promise* it externally with an SLA. The gap between the SLO and perfection is the error budget, which governs whether you can take risks (ship fast) or must slow down (stabilise).

Exam focus

Be able to state why 100% is the wrong reliability target (cost, diminishing returns, and it leaves no room to ship) and how error budgets resolve the classic dev-vs-ops tension by making "how much risk can we take" a number both sides agree on.

10.2 SLI, SLO, SLA

These three acronyms are the vocabulary of reliability. They are often confused; the exam tests the distinction directly.

SLI → SLO → SLA — and the Error Budget

Three nested concepts that turn reliability into a contract

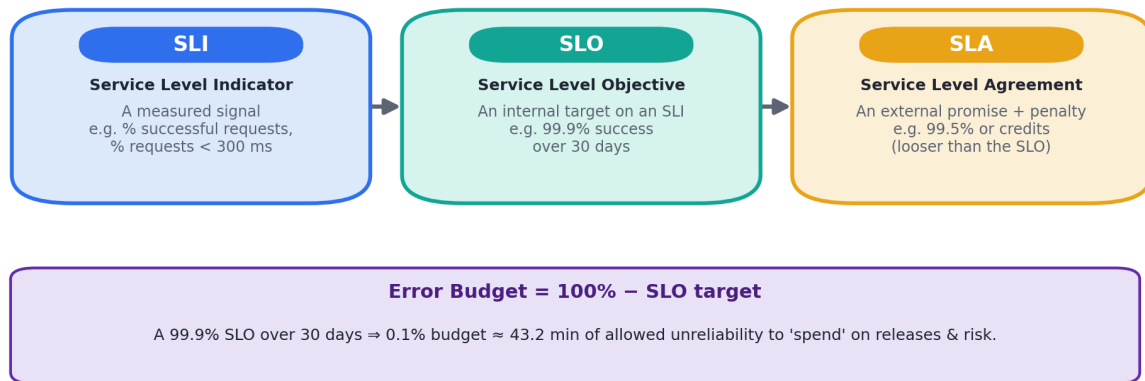


Figure 10.1 — SLI (measure) → SLO (internal target) → SLA (external promise), and the error budget derived from the SLO.

Definition — SLI, SLO, SLA

An **SLI (Service Level Indicator)** is a measured quantity of service quality — typically a ratio of good events to total events (e.g. proportion of successful requests, or of requests faster than 300 ms). An **SLO (Service Level Objective)** is the internal *target* for an SLI over a window (e.g. "99.9% over 30 days"). An **SLA (Service Level Agreement)** is an *external*, contractual promise about the SLO with consequences (credits/penalties) if missed. Rule of thumb: **SLA (loosest) < SLO (stricter internal target) — measured by the SLI**, so you catch problems before breaching the contract.

	SLI	SLO	SLA
What	A measurement	An internal target	An external contract
Example	% successful requests	99.9% over 30 days	99.5% or service credits
Audience	Engineers	Eng / product	Customers / legal
Consequence of miss	—	Spend error budget; freeze releases	Financial penalty
Strictness	—	Stricter than SLA	Loosest (with margin)

Definition — Datadog SLO types

Datadog implements SLOs two ways. A **metric-based SLO** computes reliability as $\text{sum of good events} \div \text{sum of total events}$ — ideal for count data and needs no monitor. A **monitor-based SLO** is built on one or more existing monitors and uses their uptime — ideal when "good" is best expressed as "the monitor was OK." Both require a target (strictly below 100%) and a rolling window of 7, 30, or 90 days.

10.3 SLO design patterns

Most SLOs fall into a few shapes. In every case the recipe is the same: an SLI (a good/total ratio) + a target % + a rolling window.



Figure 10.2 — Three common SLO patterns: availability, latency, and throughput/quality.

Pattern	SLI (good ÷ total)	Example SLO
Availability SLO	successful (non-5xx) requests ÷ total	99.9% of requests succeed over 30 days
Latency SLO	requests faster than threshold ÷ total	99% of requests < 300 ms over 30 days
Throughput / quality SLO	units meeting a quality bar ÷ total	99.5% of batch jobs complete on time

Tip — Good SLOs are user-centric and few

Base SLIs on what users actually experience (successful, timely responses) rather than internal machine metrics (CPU). Keep the set small — a handful of SLOs per critical user journey beats dozens no one watches. And choose the window deliberately: shorter windows (7 days) react fast but are noisier; longer windows (90 days) are stable but forgive sustained low-grade pain.

10.4 Error budgets & burn-rate monitoring

The **error budget** is the allowed amount of unreliability — the complement of the SLO. It converts reliability into a spendable resource: as long as budget remains, teams can take risks (ship, experiment); once it's exhausted, the policy is to slow down and stabilise.

Definition — Error budget

Error budget = the gap between the SLO and 100%. A 99.9% SLO permits 0.1% unreliability; over 30 days that is ~43.2 minutes of "downtime equivalent." Datadog also expresses remaining budget as a percentage: $(\text{current status} - \text{target}) \div (100 - \text{target}) \times 100$. Because a 100% target yields a zero denominator, targets must be strictly below 100%.

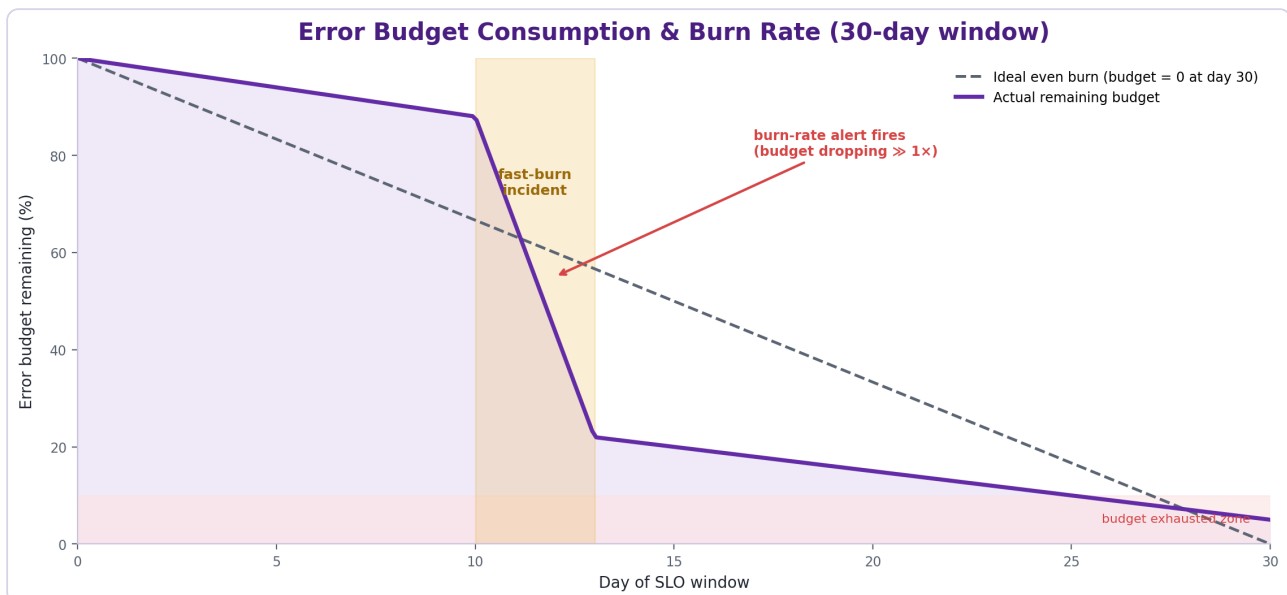


Figure 10.3 — Remaining error budget over a 30-day window. A fast-burn incident consumes budget far quicker than the ideal even rate, tripping a burn-rate alert.

Definition — Burn rate & burn-rate alerts

Burn rate is how fast you are consuming error budget relative to the sustainable pace. A burn rate of **1×** exactly exhausts the budget by the end of the window; **2×** exhausts it in half the time, and so on. **Burn-rate alerts** page when consumption is too fast — Datadog's SLO burn indicator uses a rolling 2-hour window (elevated at 1–6×, critical above 6×). A robust setup uses **multi-window, multi-burn-rate** alerting: a **fast-burn** rule (high rate over a short window) pages on acute outages, while a **slow-burn** rule (lower rate over a long window) catches chronic, low-grade erosion.

Alert type	Detects	Window / rate (illustrative)	Urgency
Fast burn	Acute outage eating budget quickly	~14× over 1 hour	Page immediately
Slow burn	Chronic degradation	~3–6× over 6–24 hours	Ticket / next business day

Exam focus

Know that error budget = 100% – SLO, that a 1× burn rate exactly spends the budget over the window, and why multi-window burn-rate alerting is superior to a single threshold: it pages loudly for fast burns while still catching slow burns, without flooding you with noise. Tie this back to Day 02: burn-rate alerting is how SLOs become actionable monitors.

10.5 Hands-on labs, use cases & review questions

Use case — Spending the budget wisely

A team's checkout service has a 99.9% availability SLO (30-day window). Three weeks in, the budget dashboard shows 60% of the budget still remaining and burn well under 1×. Product wants to ship a risky payments refactor. Because there is ample budget and no active fast burn, the SRE lead approves the release behind a feature flag — the error budget *authorises* the risk. Two weeks later, after an unrelated

incident consumes most of the budget, a second risky change is deferred until the window resets. *Lesson:* the error budget turned "should we ship?" from an argument into a data-driven policy.

Lab 10.1 — Create an availability SLO

Goal: define a correct availability SLO end to end.

For the `checkout` service, design an availability SLO: choose the SLI (good/total definition), the SLO type (metric- vs monitor-based), the target, and the window. Then compute the monthly error budget in minutes and state the release policy when the budget is exhausted.

Solution

SLI: good = non-5xx (successful) checkout requests; total = all checkout requests; SLI = good ÷ total.

Type: *metric-based* (count data, no monitor needed) using APM trace metrics for hits and errors.

Target: 99.9%; **window:** rolling 30 days (set the SLO stricter than any customer SLA). **Error budget:** 0.1% of 30 days = $0.001 \times 43,200 \text{ min} \approx 43.2 \text{ minutes}$ of allowed failure-equivalent per window.

Policy: while budget remains, releases proceed; once exhausted, freeze non-critical releases and prioritise reliability work until the rolling window recovers budget. Add fast- and slow-burn alerts so you're warned long before exhaustion.

Lab 10.2 — Create a latency SLO

Goal: express a performance target as an SLO.

Turn the requirement "checkout should feel fast" into a latency SLO. Define the SLI precisely, pick a target and window, and explain how it differs structurally from the availability SLO.

Solution

SLI: good = requests with response time < threshold (say 300 ms); total = all requests; SLI = fast ÷ total. **SLO:** 99% of checkout requests < 300 ms over 30 days. **Structural difference:** an availability SLO counts *success/failure*; a latency SLO counts requests *above/below a latency threshold* — so you must choose that threshold (informed by the Day 02 percentile analysis and the Day 03 Apdex T). Note the link: "99% < 300 ms" is essentially a p99 latency target expressed as an SLO. Pick the threshold from user expectation, not current performance, so the SLO stays meaningful.

Lab 10.3 — Build an error-budget dashboard & burn alerts

Goal: operationalise SLOs with a dashboard and the right alerts.

Design an error-budget dashboard for the two SLOs above and specify the burn-rate alerts. What two burn rules would you configure, and how do their urgencies differ?

Solution

Dashboard: per SLO, show current SLI vs target (status), remaining error budget (% and minutes), a burn-down timeseries over the window (like Figure 10.3), and current burn rate. Add a table of recent budget-consuming incidents. **Alerts (multi-window, multi-burn-rate):** (1) a *fast-burn* rule — e.g. burn rate $\approx 14\times$ over 1 hour $\rightarrow$ **page** immediately (an acute outage will exhaust the month's budget in hours). (2) a *slow-burn* rule — e.g. $\approx 3\times$ over 6 hours $\rightarrow$ **ticket** / non-paging (chronic erosion that should be fixed but isn't an emergency). This pair maximises signal: loud for real fires, quiet-but-tracked for slow leaks.

Review questions — Module 10

- Q1. Why is 100% the wrong reliability target?
- Q2. Define SLI, SLO, and SLA, and give their correct strictness ordering.
- Q3. How do error budgets resolve the dev-vs-ops tension?
- Q4. Distinguish metric-based from monitor-based SLOs in Datadog.
- Q5. What three ingredients define any SLO?
- Q6. Compute the monthly error budget (in minutes) for a 99.95% SLO over 30 days.
- Q7. What does a burn rate of $1\times$ mean? Of $4\times$?
- Q8. Why use multi-window, multi-burn-rate alerting instead of a single threshold?
- Q9. Why should an SLO be set stricter than the customer-facing SLA?

Answers in Appendix A.

Intelligent Alerting & AIOps

Alert on what matters and nothing else: evolve from static thresholds to anomaly detection and forecasting, then let AIOps and Watchdog reduce noise, correlate incidents, and find root causes.

11.1 Alert engineering: from static to predictive

Good alerting is a design problem. The goal is high signal and low noise: page a human only when action is genuinely needed. Alert sophistication has evolved through four stages, each reducing false alarms and warning earlier.

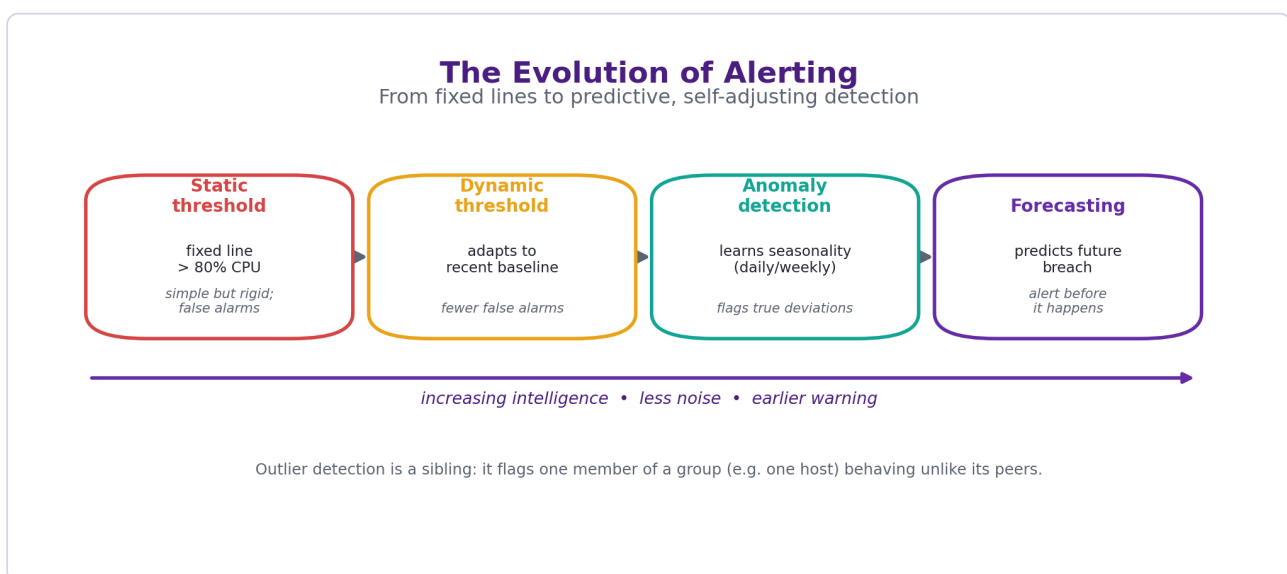


Figure 11.1 — From static thresholds to forecasting: increasing intelligence, less noise, earlier warning.

Monitor type	How it decides	Best for
Static threshold	Fires when a metric crosses a fixed line (e.g. CPU > 80%)	Hard limits with a clear "never exceed" value
Dynamic threshold	Compares against a recent adaptive baseline	Metrics whose "normal" drifts over time
Anomaly detection	Learns seasonality (daily/weekly patterns) and flags true deviations	Metrics with strong time-of-day/day-of-week cycles (traffic, orders)
Forecasting	Projects the trend and alerts <i>before</i> a predicted breach	Slow-moving resources (disk filling, quota approaching)
Outlier detection	Flags one group member behaving unlike its peers	Fleets — one host/pod misbehaving among many

Definition — Anomaly vs forecast vs outlier

Anomaly detection asks "is this metric abnormal *for this time of day?*" (seasonality-aware). **Forecasting** asks "will this metric breach a threshold *soon?*" (predictive, for trends). **Outlier detection** asks "is one member of this group behaving unlike the others *right now?*" (peer comparison across a fleet). Choosing the right one is a common exam and real-world decision.

Watch out — static thresholds on seasonal metrics

A fixed "orders/min > X" alert either screams every night at peak or stays silent during a quiet-hour outage. Metrics with daily/weekly rhythm need *anomaly* detection, not a static line. Conversely, don't over-engineer: a hard safety limit (disk 100%) is fine as a static/forecast alert.

Tip — Alert on symptoms (SLOs), not every cause

Prefer a small number of *symptom-based* alerts tied to user impact (SLO burn rate, error rate, latency) over hundreds of cause-based alerts (every CPU, every queue). Symptom alerts page when users hurt; causes are then found by investigation (and increasingly by Watchdog). This is the biggest single lever for cutting alert noise.

11.2 Incident analytics: noise reduction, correlation, root cause

Even well-designed alerts cluster during real incidents — one failure trips many monitors. **AIOps** (AI for IT operations) compresses that storm into a single, explained incident.

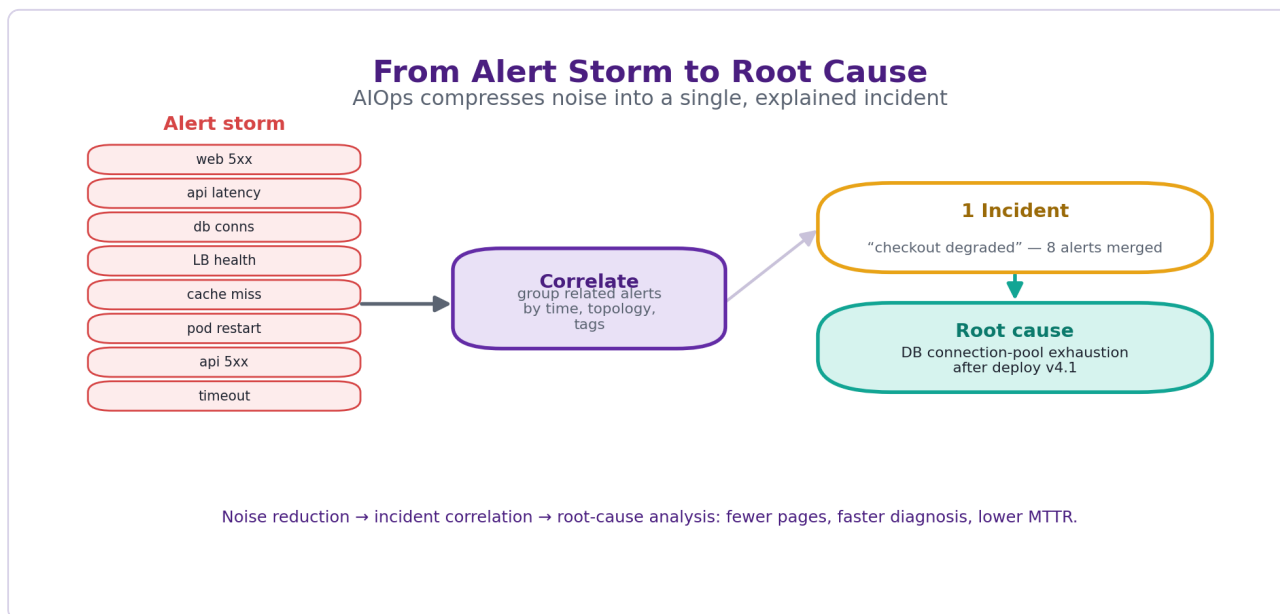


Figure 11.2 — Noise reduction → incident correlation → root-cause analysis: many alerts become one explained incident.

Capability	What it does	Payoff
Alert noise reduction	Deduplicate, group, and suppress redundant/flapping alerts	Fewer pages; less alert fatigue
Incident correlation	Group related alerts by time, topology (service map), and tags into one incident	One incident instead of 20 pages
Root cause analysis	Identify the likely originating fault (e.g. a bad deploy, a saturated dependency)	Start from a hypothesis, lower MTTR

Definition — Alert fatigue

Alert fatigue is the desensitisation that occurs when responders receive too many alerts — especially false or non-actionable ones — causing real alerts to be missed or ignored. It is the primary problem AIOps noise-reduction and correlation exist to solve, and it directly threatens MTTR (Day 02).

11.3 AIOps & Watchdog

Datadog's built-in AIOps engine is **Watchdog**. It runs continuously with no configuration, learning normal behaviour and surfacing what matters — anomalous services, faulty deployments, and dependency problems — along with their impact and likely cause.

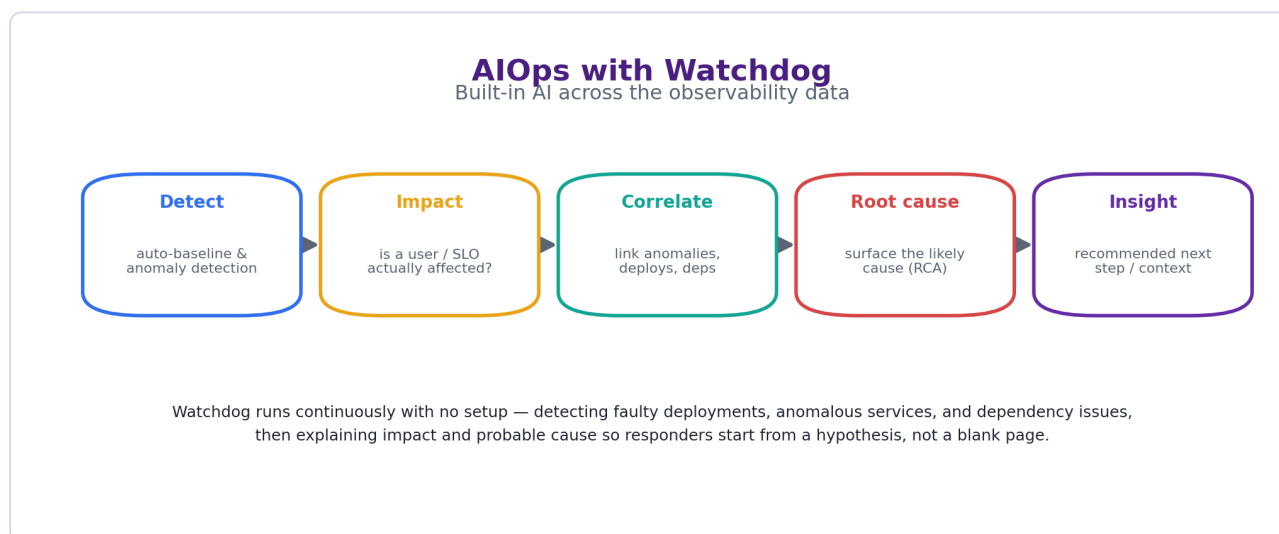


Figure 11.3 — Watchdog's flow: detect anomalies → assess impact → correlate → root cause → actionable insight.

Watchdog capability	What it provides
Automatic anomaly detection	Baselines every service/metric and flags deviations — no setup
Watchdog Insights	Context and recommendations surfaced inside the explorers
Root Cause Analysis	Points at the probable originating fault behind an anomaly
Impact analysis	Determines whether users/SLOs are actually affected (prioritisation)
Faulty deployment / dependency detection	Flags a bad release or a misbehaving dependency automatically

Exam focus

Know that Watchdog is Datadog's built-in, zero-config AIOps engine and be able to name its jobs: *detect* (anomaly), *assess impact*, *correlate*, *root-cause*, and *recommend* (insights). Its value is starting responders from a hypothesis and cutting the noise-to-signal problem that drives alert fatigue.

11.4 Hands-on lab, use cases & review questions

Use case — Silencing the 2 a.m. false alarm

A team is paged nightly because a static "requests/min < 500" alert fires every low-traffic hour — pure noise that's eroding trust in the pager. They replace it with an **anomaly** monitor that has learned the daily/weekly traffic curve, so it only fires when traffic is abnormal *for that hour*. Simultaneously they consolidate a dozen cause-based CPU/memory alerts into a single **SLO burn-rate** alert. Pages drop by 80%, and when a real off-hours outage hits, the anomaly monitor catches it (static would have stayed silent). *Lesson*: matching the monitor type to the metric's behaviour is what turns noise into signal.

Lab 11.1 — Choose the right monitor for each signal

Goal: match monitor type to metric behaviour.

Pick the best monitor type (static, dynamic, anomaly, forecast, or outlier) for each, and justify: (a) disk usage slowly climbing toward 100%; (b) orders/minute, which has strong daily and weekly cycles; (c) one pod in a 40-pod deployment showing 5× the latency of its peers; (d) a hard rule that a queue must never exceed 10,000 messages; (e) API error rate whose "normal" baseline drifts week to week.

Solution

(a) **Forecast** — predict the breach and alert before the disk fills, giving time to act. (b) **Anomaly** — seasonality-aware, so it won't false-alarm at nightly troughs or peaks. (c) **Outlier** — peer comparison flags the single misbehaving pod among its group. (d) **Static threshold** — a hard, absolute limit with a clear never-exceed value. (e) **Dynamic threshold** (or anomaly) — compares against an adaptive recent baseline rather than a fixed line. The meta-lesson: read the metric's *shape* (trending, seasonal, fleet, hard-limit, drifting) and pick accordingly.

Review questions — Module 11

- Q10.** List the alerting stages from static to predictive and what each improves.
- Q11.** When should you use anomaly detection instead of a static threshold?
- Q12.** Distinguish anomaly, forecast, and outlier detection with one use case each.
- Q13.** Why prefer symptom-based (SLO) alerts over many cause-based alerts?
- Q14.** What is alert fatigue and why does it matter for MTTR?
- Q15.** Name the three incident-analytics capabilities that turn an alert storm into one incident.
- Q16.** What is Watchdog, and what setup does it require?
- Q17.** Name four things Watchdog can do (detect/assess/correlate/root-cause/recommend).

Answers in Appendix A.

MODULE 12

Full-Stack Capstone

Prove mastery: one end-to-end incident that exercises every concept from Days 01–04, plus how to roll it all up into an executive-ready report.

12.1 Bringing it all together

Everything in this course is one correlated platform. A problem shows up as a business signal, is localised through the observability stack, explained by AI, and governed by SLOs. The capstone diagram maps how the pieces connect.

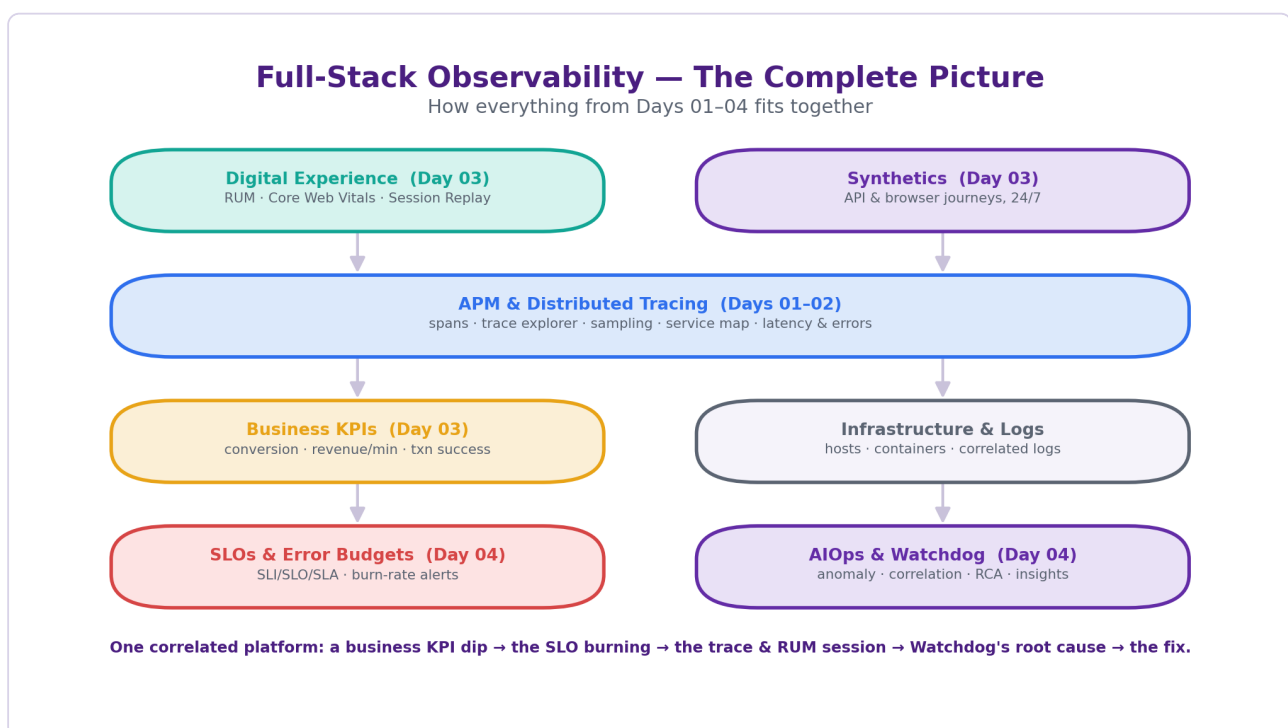


Figure 12.1 — The complete picture: digital experience, tracing, business KPIs, infrastructure, SLOs, and AIOps as one correlated platform.

Definition — Full-stack observability

The practice of correlating every layer — front-end experience (RUM), synthetic journeys, backend traces (APM), infrastructure and logs, business KPIs, SLOs, and AIOps — in a single platform, so an investigator can pivot seamlessly from a symptom at any layer to its cause at any other. The whole four-day course has been building the pieces; full-stack observability is their integration.

12.2 Capstone scenario & guided solution

The scenario

At 14:20, revenue-per-minute for your e-commerce site drops ~30%. No deploy is flagged in your change log. Support tickets mention "payment failed." Your checkout availability SLO (99.9%, 30-day) is burning fast. Walk the full-stack path from this business symptom to a resolved, reported incident — naming the tool/concept from Days 01–04 at each step.

Capstone — End-to-end incident walkthrough

Goal: integrate the entire course into one coherent response.

Produce the ordered investigation-and-response path, citing the relevant day/module at each step. Then state what you would put in the postmortem.

Solution — the full-stack path

1. **Detect (business + SLO — Day 03/04):** the drop surfaces on the business dashboard (revenue/min), and the checkout **SLO burn-rate alert** fires (fast burn), paging the on-call. Watchdog (Day 04) has also flagged an anomaly on the payments service.
2. **Localise (Synthetics + service map — Day 03/01):** the checkout **Synthetic browser test** is red from all regions; the **service map** shows `payments` red with healthy dependencies — the culprit, not a victim.
3. **Quantify real impact (RUM — Day 03):** RUM confirms real users are hitting checkout errors; segmentation shows it's all regions/devices (not a client issue); a **Session Replay** shows the failed-payment screen.
4. **Diagnose (Trace Explorer + errors — Day 02):** filter `service:payments status:error`; **Error Tracking** groups the failures into one issue (a timeout to the payment provider); the **flame graph** shows the provider call span timing out.
5. **Root cause (Watchdog + APM — Day 04/01):** Watchdog's **RCA** and the trace point to the payment provider's API latency spiking — an external dependency, not your code (explaining the absent deploy). Correlated **logs** show connection timeouts.
6. **Mitigate:** fail over to the backup payment provider / enable the circuit breaker; error rate recovers, revenue/min returns, SLO burn stops.
7. **Report & learn:** record MTTD/MTTR (Day 02), budget consumed (Day 04), and a **blameless postmortem** with an action item (add a provider-latency Synthetic + fast-burn alert so next time you detect it before revenue moves).

Why this is "full-stack": the path crossed business KPIs → SLOs → Synthetics → service map → RUM → traces → errors → Watchdog → logs — every layer of the course, on one timeline, each pivot narrowing scope from *symptom* to *cause* to *fix*.

12.3 Executive reporting

The last mile of reliability engineering is communicating it upward. Executives don't want span waterfalls; they want outcomes, risk, and where to invest. Translate the technical story into their language.

Engineers see...	Executives need...
p99 latency, error rate, flame graphs	Customer impact and revenue at risk
SLO burn-rate, error budget %	"Are we meeting our reliability commitments?" (SLO attainment vs target)
MTTD / MTTR per incident	Trend in incident frequency & recovery speed (are we improving?)
Individual alerts	Top risks and the investment needed to reduce them

Tip — A good executive reliability report

Lead with **SLO attainment** (met/missed vs target, per critical service) and **business impact** (revenue/customers affected), show the **trend** in reliability KPIs (MTTR, incident count, error-budget consumption) over the quarter, and end with a short list of **prioritised actions and their expected payoff**. One page, outcome-first, no jargon. The SLO is the perfect executive metric because it is a single number that ties engineering effort to a business commitment.

Exam focus

Be able to translate a technical signal into a business statement (e.g. "p99 breached" → "the slowest 1% of customers experienced >2s checkouts, correlating with a 3% conversion drop") and to name what belongs in an executive report: SLO attainment, business impact, reliability trends, and prioritised investments.

Appendix A — Answer keys

Module 10

#	Answer
Q1	It's prohibitively expensive, has diminishing returns (users can't perceive it), and leaves no room to ship changes; a target below 100% creates a manageable error budget.
Q2	SLI = a measured signal (good/total ratio); SLO = internal target on an SLI; SLA = external contractual promise with penalties. Strictness: SLA (loosest) < SLO (stricter internal) < measured by SLI.
Q3	They make "how much risk can we take" a shared number: while budget remains, dev may ship; when it's exhausted, the policy is to stabilise — so both teams optimise toward the same target.
Q4	Metric-based = good events ÷ total events (count data, no monitor); monitor-based = uses one or more monitors' uptime (time-based). Both need a sub-100% target and a 7/30/90-day window.
Q5	An SLI (good/total ratio) + a target percentage + a rolling time window.
Q6	0.05% of 30 days = $0.0005 \times 43,200 \text{ min} \approx \mathbf{21.6 \text{ minutes}}$.
Q7	1× consumes the entire budget exactly over the window; 4× consumes it in a quarter of the window (four times too fast).
Q8	A fast-burn rule pages loudly on acute outages while a slow-burn rule catches chronic erosion — together they page for real emergencies without flooding you, which a single threshold can't balance.
Q9	To keep a safety margin: you want to detect and fix problems (burning the internal budget) before you breach the customer contract and incur penalties.

Module 11

#	Answer
Q10	Static (fixed line) → dynamic (adaptive baseline) → anomaly (seasonality-aware) → forecasting (predict breach). Each reduces false alarms and/or warns earlier.
Q11	When the metric has strong seasonality (daily/weekly cycles) so a fixed line would false-alarm at peaks or miss off-hours problems.
Q12	Anomaly: abnormal-for-this-time (e.g. orders/min). Forecast: will-breach-soon (e.g. disk filling). Outlier: one-of-a-group-differs (e.g. one slow pod among 40).
Q13	Symptom (SLO) alerts fire only when users are actually impacted, so there are few and they're actionable; cause alerts are numerous and mostly non-actionable, driving noise. Find causes by investigation/Watchdog.
Q14	Desensitisation from too many (often false) alerts, causing real alerts to be missed/ignored — it raises MTTD/MTTR because responders react slower or not at all.
Q15	Alert noise reduction (dedupe/suppress), incident correlation (group by time/topology/tags), and root-cause analysis.
Q16	Datadog's built-in, zero-configuration AIOps engine that continuously baselines behaviour and surfaces anomalies, impact, and likely causes. No setup required.
Q17	Any four: automatic anomaly detection, impact analysis, incident correlation, root-cause analysis, faulty-deployment/dependency detection, Watchdog Insights/recommendations.

Appendix B — SRE & alerting cheat sheet

Reliability vocabulary

Term	Definition
SLI	Measured service-quality signal ($\text{good} \div \text{total events}$)
SLO	Internal target on an SLI over a window (7/30/90 days), $< 100\%$
SLA	External contractual promise with penalties (looser than the SLO)
Error budget	$100\% - \text{SLO target}$; the allowed unreliability to "spend"
Burn rate	Speed of budget consumption; $1\times$ exhausts it exactly over the window
Fast / slow burn	Acute (page) vs chronic (ticket) budget consumption
Metric-based SLO	$\text{good events} \div \text{total events}$; no monitor needed
Monitor-based SLO	Based on monitor uptime
Toil	Repetitive manual operational work to be automated away
Blameless postmortem	Incident review focused on systems/learning, not blame

Error budget quick reference (30-day window)

SLO target	Error budget	$\approx \text{time} / 30 \text{ days}$
99%	1%	~7.2 hours
99.9%	0.1%	~43.2 minutes
99.95%	0.05%	~21.6 minutes
99.99%	0.01%	~4.3 minutes

Monitor-type selector

Metric shape	Use
Hard never-exceed limit	Static threshold
Baseline drifts over time	Dynamic threshold
Daily/weekly seasonality	Anomaly detection
Trending toward a limit	Forecasting
One of many peers differs	Outlier detection
User-impact symptom	SLO burn-rate alert

One-page memory hooks

SRE: 100% is wrong; target < 100% = error budget. **Trio:** SLI measures, SLO targets, SLA promises (SLA loosest). **Budget:** 100% – SLO; 1× burn spends it over the window; fast-burn pages, slow-burn tickets.

Alerting: static → dynamic → anomaly → forecast (+ outlier for fleets); alert on symptoms/SLOs, not every cause. **AIOps/Watchdog:** detect → impact → correlate → root-cause → insight; kills alert fatigue.

Capstone: business dip → SLO burn → Synthetics/service map → RUM → traces/errors → Watchdog RCA → fix → executive report.

Appendix C — Course wrap-up & sources

You've completed the four-day path: from distributed tracing fundamentals (Day 01), through trace analytics and KPIs (Day 02), business observability and digital experience (Day 03), to reliability engineering, AIOps, and executive reporting (Day 04). Together these cover the full arc — instrument, collect, analyse, quantify, experience, govern, and communicate.

The four days at a glance

Day	Focus	Key ideas
01	APM & distributed tracing	spans/traces, agent pipeline, sampling, service map
02	Trace analytics & KPIs	Trace Explorer, percentiles, errors, four KPI families
03	Business obs & digital experience	business KPIs, RUM & Core Web Vitals, Synthetics
04	Reliability, AIOps & reporting	SLI/SLO/SLA, error budgets, Watchdog, capstone

Sources for Day 04 (confirm current defaults against the live docs):

- Datadog Docs — Service Level Objectives: docs.datadoghq.com/service_management/service_level_objectives/
- Datadog Docs — SLO error budget & burn-rate alerts: docs.datadoghq.com/service_management/service_level_objectives/burn_rate/
- Datadog Docs — Monitor types (anomaly, forecast, outlier): docs.datadoghq.com/monitors/types/
- Datadog Docs — Watchdog: docs.datadoghq.com/watchdog/
- Datadog Docs — Incident Management & Event correlation: docs.datadoghq.com/service_management/
- Google — Site Reliability Engineering (SRE Book & Workbook): SLOs, error budgets, multi-window burn-rate alerting.

Day 04 study guide compiled August 2026. Diagrams are original, produced for this guide. UI screenshots are intentionally left as labelled placeholders to capture live from your own Datadog account. Module 12 (Full-Stack Capstone) is included to fulfil the "3 modules / full-stack capstone" framing of the Day 04 outline.